Toegepaste Informatica

1TI/ACS

Front-End Development

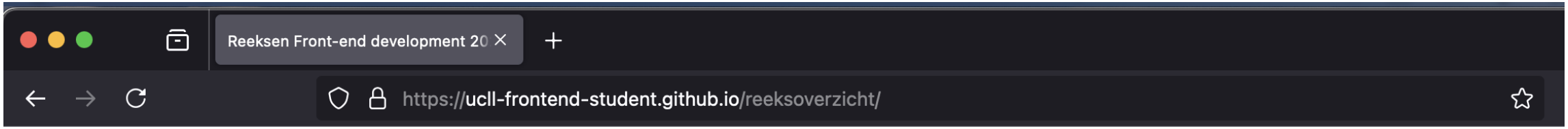
JS: Client Server Interaction; Asynchroon JS

J. Pieck, L. Van Houdt, T. Van Nuland, S. Van Peborgh

Client Server Model

- Static Web Page: retrieving HTML
- Protocol – Request – Response
- Back End: retrieving and storing data (JSON)

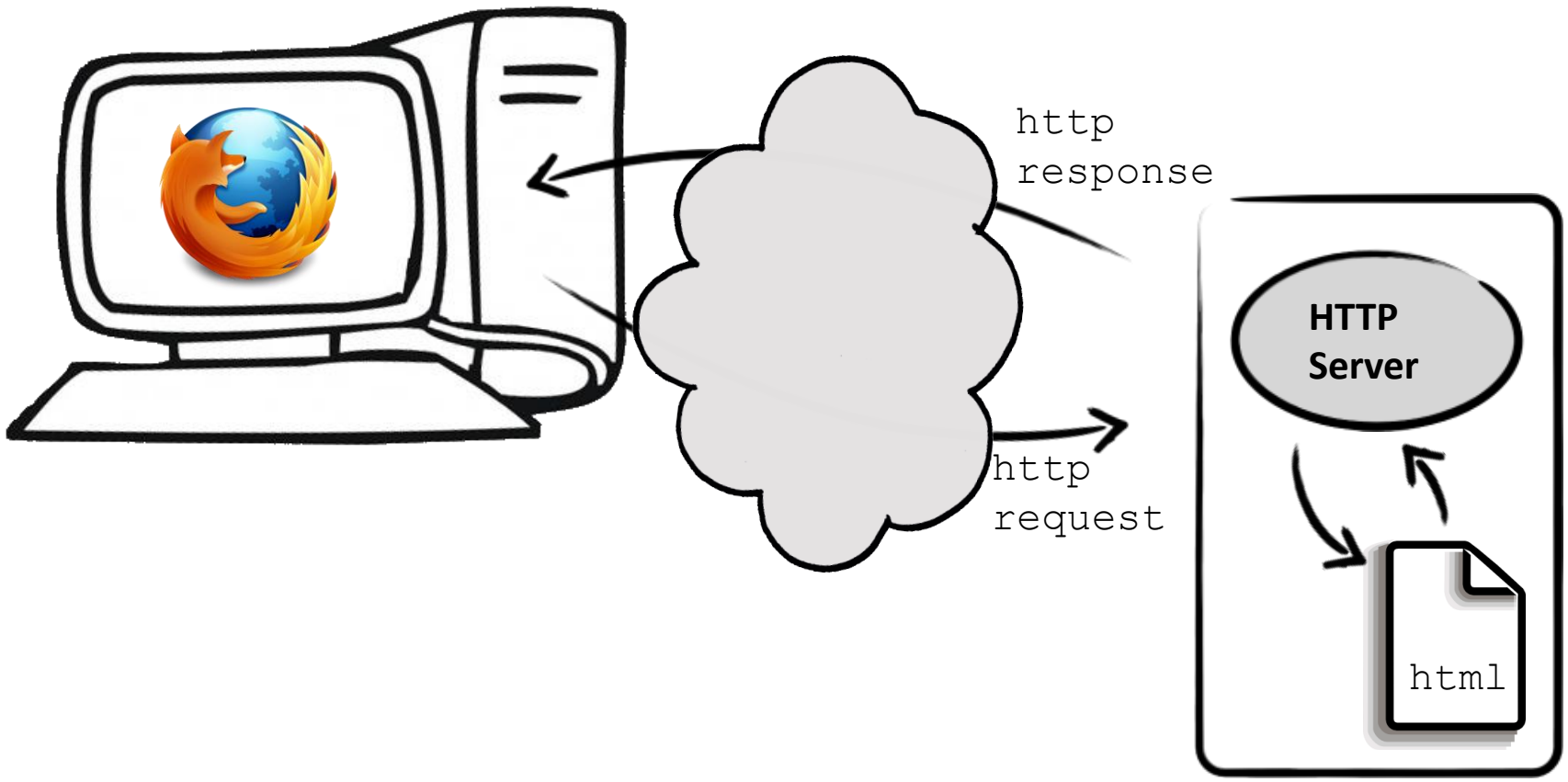
How to get this static page on my PC?



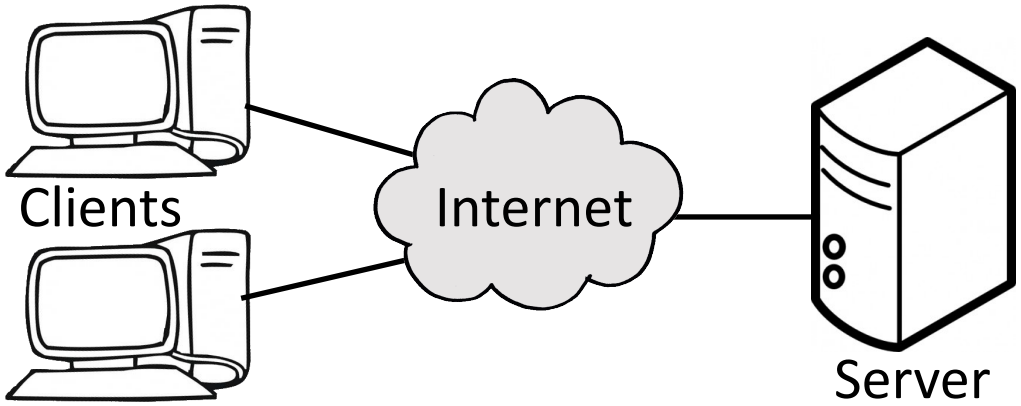
Overzicht reeksen Front-end Dev 2024-25

- Reeks 1 (lector: Leen Van Houdt)
- Reeks 2 (lector: Tim Destexhe)
- Reeks 3 (lector: Tim Destexhe)
- Reeks 4 (lector: Greetje Jongen)
- Reeks 5 (lector: Leen Van Houdt)
- Reeks 6 (lector: Tim Destexhe)
- Reeks 7 (lector: Leen Van Houdt)
- Reeks 8 (lector: Greetje Jongen)

```
http://<host>/<path>
```



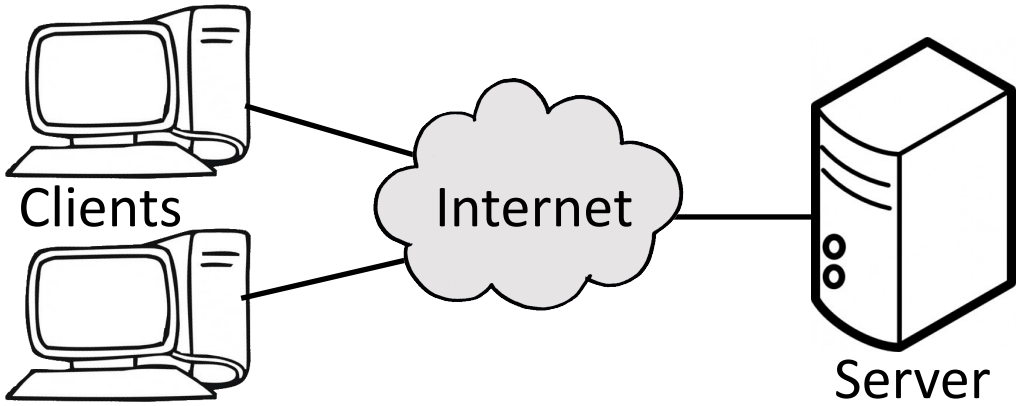
Client Server Model



The **client-server** model is a software architecture model consisting of two parts: **client** systems and **server** systems, both using a computer network or on the same computer.

The **client** process always **starts** a connection to the server, while the **server** process always **waits for requests** from any client.

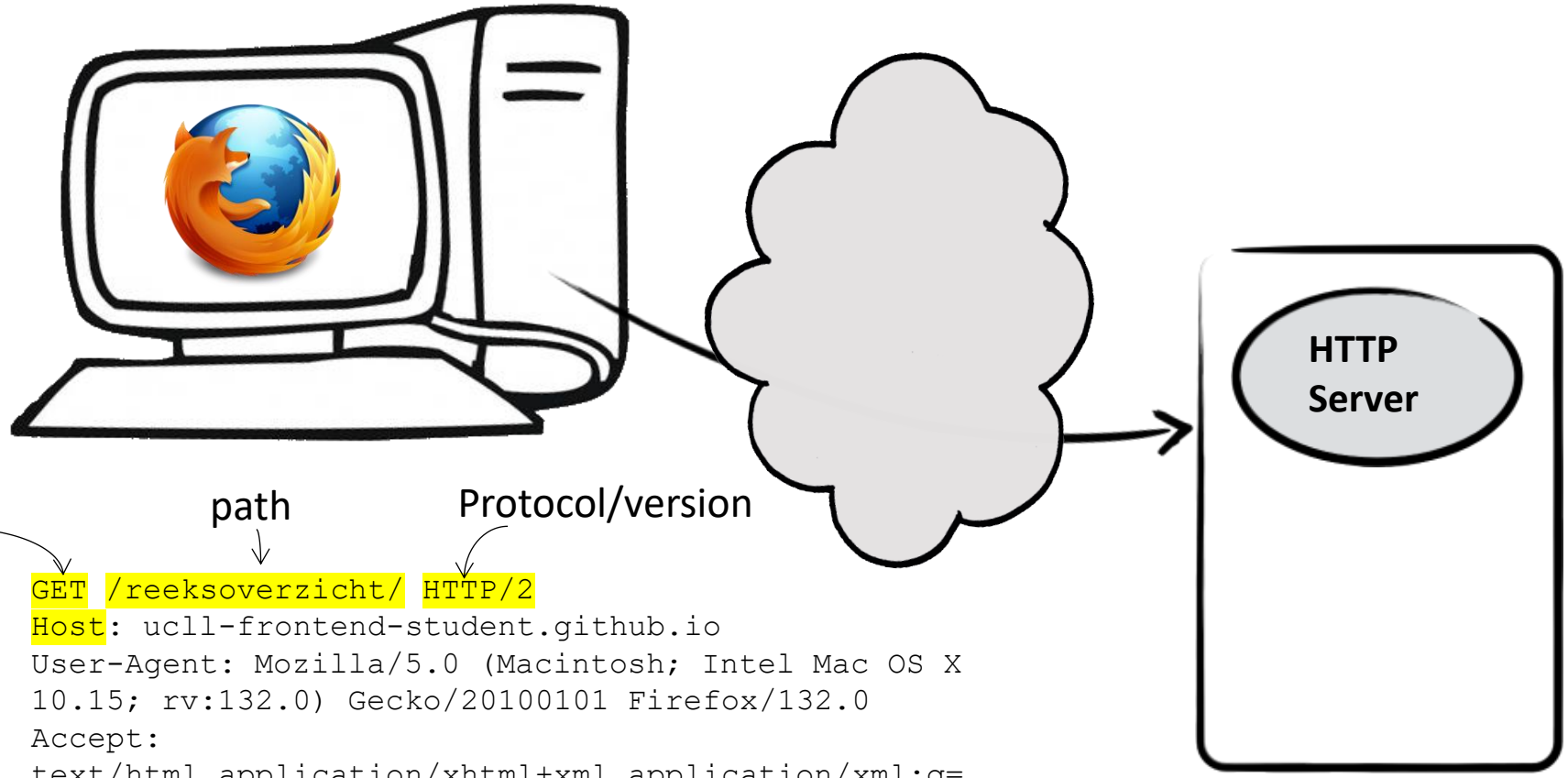
Client Server Model



Needs a service	Offers a service
Runs client software	Runs server software
Shares no resources	Shares resources with client
Starts the communication	Waits for request
Sends request	Gives response

https://ucll-frontend-student.github.io/reeksoverzicht/

REQUEST



method

path

Protocol/version

headers

```
GET /reeksoverzicht/ HTTP/2
Host: ucll-frontend-student.github.io
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X
10.15; rv:132.0) Gecko/20100101 Firefox/132.0
Accept:
text/html,application/xhtml+xml,application/xml;q=
0.9,*/*;q=0.8
Accept-Language: nl,en-US;q=0.7,en;q=0.3
Accept-Encoding: gzip, deflate, br, zstd
Connection: keep-alive
```

https://ucll-frontend-student.github.io/reeksoverzicht/

RESPONSE

Protocol/version

status

headers

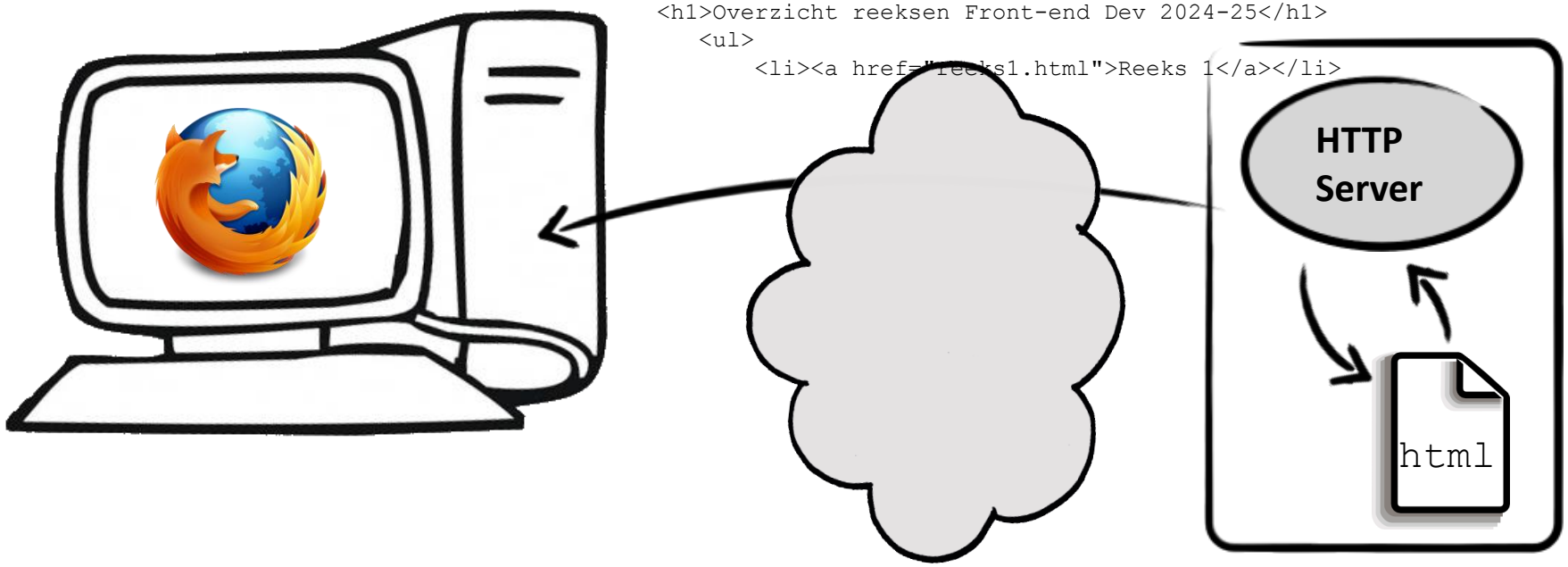
```
HTTP/2 200
server: GitHub.com
content-type: text/html; charset=utf-8
permissions-policy: interest-cohort=()
last-modified: Wed, 09 Oct 2024 07:23:11 GMT
access-control-allow-origin: *
strict-transport-security: max-age=31556952
…

<!DOCTYPE html>
<html lang="nl">

<head>
  …
</head>

<body>
 <h1>Overzicht reeksen Front-end Dev 2024-25</h1>
    <ul>
        <li><a href="reeks1.html">Reeks 1</a></li>
```

body



Firefox Developer Edition Bestand Bewerken Beeld Geschiedenis Bladwijzers Extra Venster Help Di 22 okt 16:13

Reeksen Front-end development 20

https://ucll-frontend-student.github.io/reeksoverzicht/

reeksen | Reeks 2 | Reeks 4 | 3 manieren om kant... | AO-project-book-te... | LF6 (van W naar O) ... | Swagger UI | localhost | Localhost8080 | Homepage 3461_re... | Andere bladwijzers

Overzicht reeksen Front-end Dev 2024-25

- Reeks 1 (lector: Leen Van Houdt)
- Reeks 2 (lector: Tim Destexhe)
- Reeks 3 (lector: Tim Destexhe)
- Reeks 4 (lector: Greetje Jongen)

Inspector | Console | Debugger | Netwerk | Stijleditor | Prestaties | Geheugen | Opslag | 1

URL's filteren | Buffer uitschakelen | Geen beperking

Alle | HTML | CSS | JS | XHR | Lettertypen | Afbeeldingen | Media | WS | Anders

St	Me	Do...	Bestand	Initi...	Ty	Ove...	Gr
200	GE	u...	/reeksoverzicht/	doc...	htr	1,31 ...	1,4
200	GE	f...	css?family=Noto+S	styl...	css	1,71 ...	13,
404	GE	u...	favicon.ico	Favi...	htr	5,79...	9,1
200	GE	f...	ga6daw1J5X9T9RW	font	wo	43,9...	43
200	GE	f...	ga6daw1J5X9T9RW	font	wo	43,9...	43

5 aanvragen | 110,80 kB / 96,77 kB overgebracht

Headers | Cookies | Aanvraag | Antwoord | Timings | Beveiliging

Headers filteren | Blokkering | Opnieuw verzenden

GET https://ucll-frontend-student.github.io/reeksoverzicht/

Status 200
Versie HTTP/2
Overgebracht 1,31 kB (grootte 1,40 kB)
Prioriteit aanvragen Highest
DNS-omzetting Systeem

Antwoordheaders (747 B) | Onbewerkt

accept-ranges: bytes
access-control-allow-origin: *

Client Server Model

- Static Web Page: retrieving HTML
- Protocol – Request – Response
- Back End: retrieving and storing data (JSON)

HTTP

- Protocol for fetching resources such as HTML documents
- Client-server protocol
  - requests are initiated by the client
- Stateless
  - no link between two requests being sucessively carried out on the same connection
- Requests & responses consist of a body and a header
  - The header contains metadata
    - Who executed this request?
    - In what format does the sender of the request expect a response from the backend?
    - …
  - The body contains the data of the request or response

https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview

HTTP — Request Methods

- The method indicates what type of request is being sent to the URL
  - GET: retrieving information
  - POST: storing new information
  - PUT: updating existing information
  - DELETE: removing information

- Other, more specialised methods exist
  - https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods

- The combination of method and URL must be unique within the backend

HTTP— Response Status Codes

- The status code of the response indicates whether the backend could succesfully process the request
  - 2xx – Succes
  - 3xx – Redirect
  - 4xx – Client Error
  - 5xx – Server Error

- Other, more specialised status codes exist
  - https://en.wikipedia.org/wiki/List_of_HTTP_status_codes

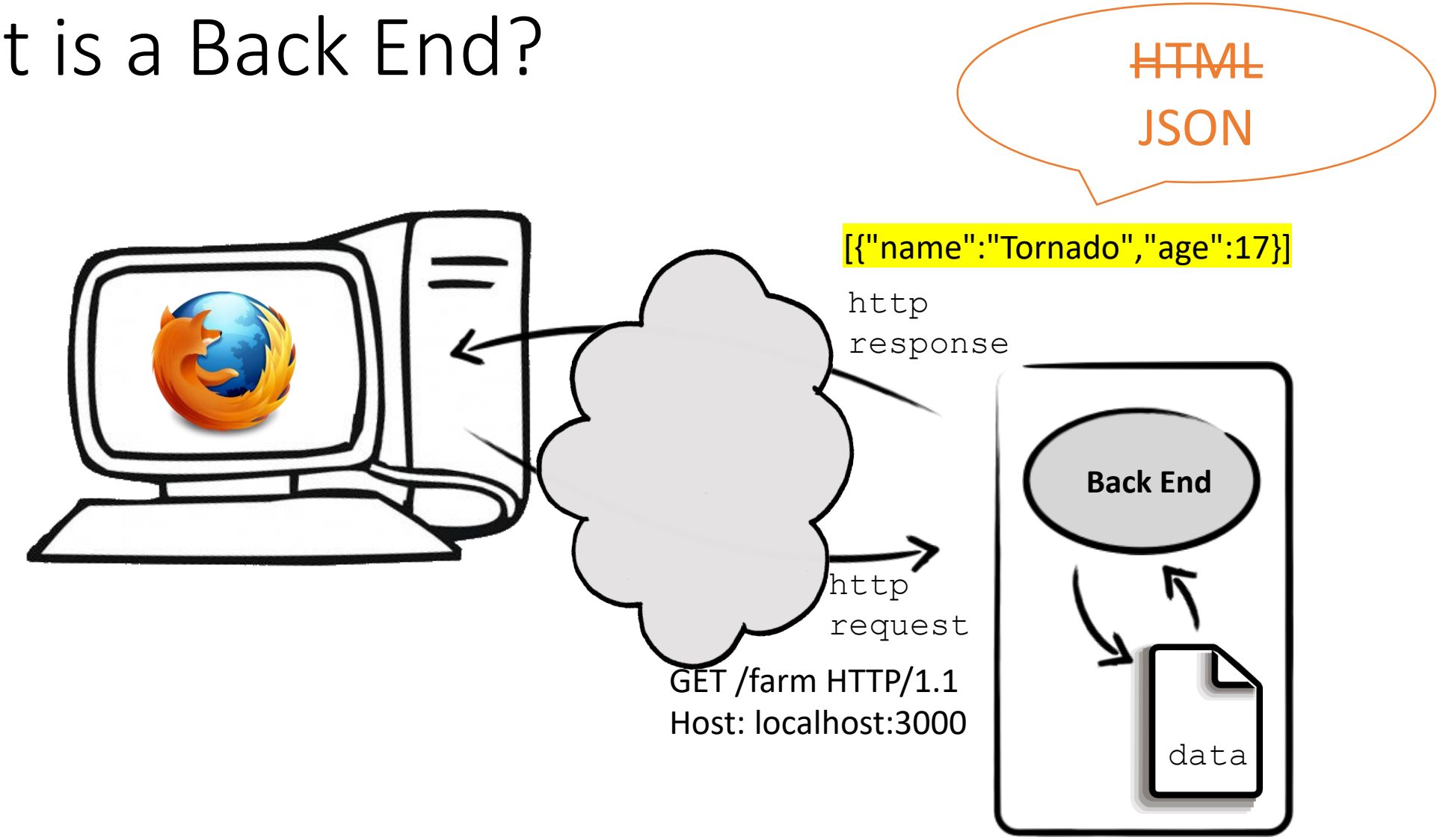
Client Server Model

- Static Web Page: retrieving HTML
- Protocol – Request – Response
- Back End: retrieving and storing data (JSON)

What is a Back End?

- Frontend applications need data to function correctly
  - This data is often dynamic and cannot be bundled together with the frontend
- So the data needs to be retrieved from another software application
  - This application functions as a *back end* for the frontend
- A back end is a software application that
  - Stores and retrieves data (most of the time using a database)
  - Guards application rules and constraints
  - Enables the user-facing applications to function correctly

What is a Back End?



HTTP REQUEST RESPONSE BODY

- The body of the request and response contains its actual data
- It is structured with **J**ava**S**cript **O**bject **N**otation (JSON)
  - Describes information in a textual format, similar to XML
  - Hierarchical structure
  - Designed to have less overhead as XML
  - Closely linked to JavaScript

JSON Object

- Indicated with {}
- Unsorted set of key/value pairs
- Key and value are separated by :
- Key/value pairs are separated by ,

```
{
 "book":{
  "title": "Lord of the Rings",
  "author": "JRR Tolkien"
 }
}
```

Object

Key

Value

JSON Array

- Indicated with []
- Sorted list of values
- Values are separated by ,

```
{
 "book":[“Lord of the Rings",“Harry Potter"]
}
```

Array Value Value

Async JavaScript

- The method fetch()
- Async/await

fetch()

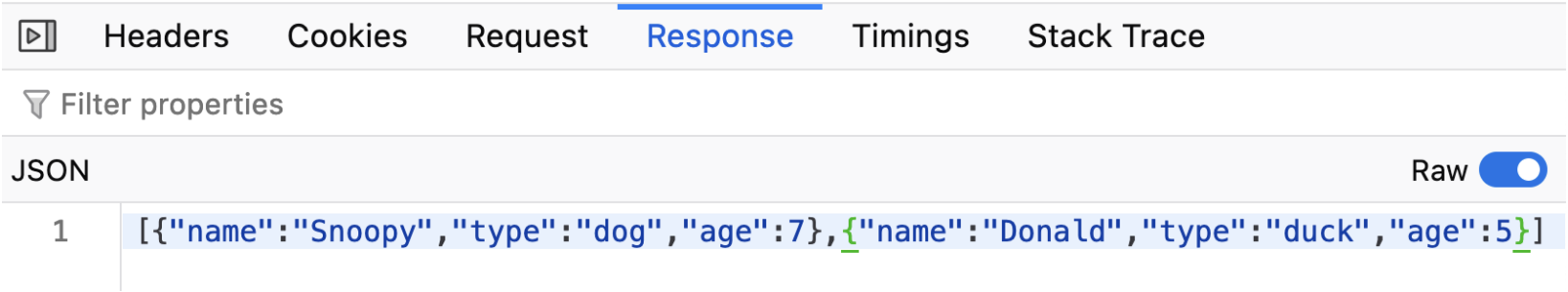
- Makes HTTP request
  - Can include headers and body
- Body data is in JSON (request & response)
- Processes response
  - Returns a *Promise* as long as the request has not been completed (a ‘promise’ that something will follow)
- Asynchronous function

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

Example: fetch() with GET request

```
fetch("http://localhost:3000/animals");
```

Response body

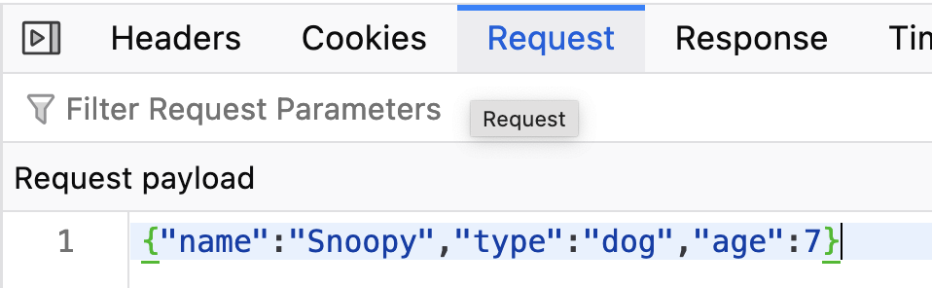


Example: fetch() with POST request

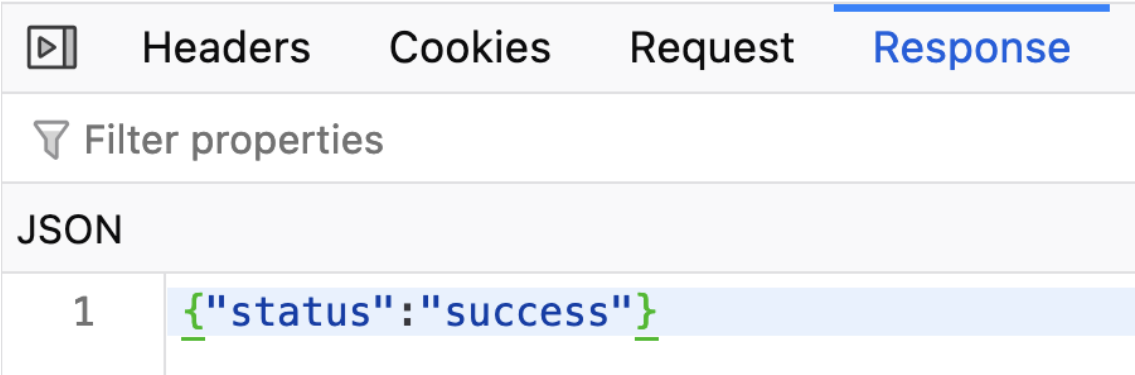
```
const animal = { name: "Snoopy", type: "dog", age: 7 };
fetch("http://localhost:3000/animals", {
        method: "POST",
        headers: {
                Accept: "application/json",
                "Content-Type": "application/json"},
        body: JSON.stringify(animal)});
```

Convert JS object to JSON object

Request body



Response body



Async JavaScript

- The method fetch()
- Async/await

Asynchronous Function: Problem

```javascript
let a = 1;
console.log(`a = ${a}`);
fetch('./index.html').then(() => {
    console.log("HTML page is fetched!");
});
a++;
console.log(`a second time = ${a}`);
```

```
a = 1
a second time = 2
HTML page is fetched!
```

- The function fetch is an *asynchronous* function.
- JS normally runs over the lines of a script from top to bottom.
- Asynchronous functions are first put in a separate queue and are executed after the rest.

Asynchronous Function: async/await

- The function fetch() is async and returns a *promise*.
- An *await* causes the code to pause for a moment until that *promise* is ready (completed or rejected).
- The await keyword is only valid inside async functions. Therefore:
  - code is clustered in myFunction()
  - myFunction() is labeled as asynchronous function with keyword *async*

```
const myFunction = async () => {
        let a = 1;
        console.log(`a = ${a}`);
        await fetch('./index.html').then(() => {
        console.log("HTML page is fetched!");});
        a++;
        console.log(`a second time = ${a}`);}
myFunction();
```

```
a = 1
HTML page is fetched!
a second time = 2
```

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function#description

Example: fetch() with GET request

```
const animals = [];
const fetchAnimals = async () => {
        const response = await fetch("http://localhost:3000/animals");

        const result = await response.json();
        animals.push(...result);
};
```

Convert JSON object to JS array

HTML: Form

Form: Syntax

```
<form id=”myForm" method=”POST” action="doSomething">
<p>
<label for="name">Name:</label>
<input id="name" type="text">
</p>
<p>
<input id="submit" type="submit" value="Add">
</p>
</form>
```

Form: Default behaviour

- User clicks on submit button (input field)
- Browser sends HTTP request as defined in <form> attributes
  - Method: specifies the HTTP method to be used when submitting form data
  - Action: defines the action to be performed

Manipulating default behaviour with JS

- Add eventlistener on <form> not on <input type="submit">
- Tell user agent that the default action should not be taken with

  event.preventDefault()
- Define your own action

Example

```
document
        .getElementById(”myForm")
        .addEventListener(
                "submit",
                (event) => {
                        event.preventDefault();
                        addAnimal()
                }
        )
```